ABAP - Proyecto: Aeronave

Alberto Piedra Venegas

Noviembre 2026

Índice

1. Proyecto ABAP orientado a objetos **2**
1.1. Introducción y caso de uso . . . 2
1.2. Requisitos de la aplicación . . . 2
1.3. Entorno de desarrollo . . . 3

2. Diseño e implementación **4**
2.1. Jerarquía de clases . . . 4
2.2. Clase padre: ZCL_AERONAVE_12 . . . 4
2.2.1. Constantes y atributos . . . 4
2.2.2. Constructor y contador de aeronaves . . . 5
2.2.3. Registro de vuelos y revisiones . . . 5
2.2.4. Estado de revisión . . . 6
2.2.5. Ficha, vuelos hasta revisión y coste . . . 6
2.3. Clase hija: ZCL_AVION_CARGA_12 . . . 7
2.4. Clase hija: ZCL_AVION_PASAJEROS_12 . . . 8
2.5. Clase de taller: ZCL_TALLER_12 . . . 9
2.6. Clase de utilidades: ZCL_AERO_UNIDADES_12 . . . 10

3. Pruebas **12**
3.1. Estrategia de pruebas . . . 12
3.2. Prueba de la clase padre . . . 12
3.3. Prueba del avión de carga . . . 13
3.4. Prueba del avión de pasajeros . . . 13
3.5. Prueba del taller . . . 14
3.6. Prueba de las utilidades . . . 15
3.7. Escenario completo: flota y taller . . . 15

4. Incidencias detectadas y corrección **15**

5. Conclusiones **17**

1. Proyecto ABAP orientado a objetos

1.1. Introducción y caso de uso

En este caso práctico construiremos un pequeño sistema de gestión de flota para una compañía aérea utilizando ABAP orientado a objetos. El sistema controla las horas de vuelo de cada aeronave, avisa cuando se acerca la revisión obligatoria, impide volar a los aviones que la tienen pendiente, calcula costes operativos y registra las intervenciones de un taller de mantenimiento.

El proyecto se ha desarrollado en Eclipse con ABAP Development Tools (ADT), dentro del paquete `ZPROYECTO_AERONAVE_12`, que a su vez forma parte del paquete de ejercicios `ZPOO_12`. Contiene 11 clases: seis clases de negocio y cinco clases de prueba.

El recorrido que seguiremos es: clase padre *Aeronave*, clases hijas de carga y de pasajeros, clase de taller, clase de utilidades y, por último, las clases de prueba que comprueban el comportamiento de todo el conjunto.

1.2. Requisitos de la aplicación

El proyecto está formado por las siguientes clases:

Clase	**Tipo**	**Responsabilidad**
`ZCL_AERONAVE_12`	Clase padre	Horas de vuelo, revisiones, coste base y contador de aeronaves.
`ZCL_AVION_CARGA_12`	Clase hija	Coste de bodega, recargo por mercancía y validación de carga.
`ZCL_AVION_PASAJEROS_12`	Clase hija	Coste de cabina, plazas y porcentaje de ocupación.
`ZCL_TALLER_12`	Clase de negocio	Registro y consulta de intervenciones de mantenimiento.
`ZCL_AERO_UNIDADES_12`	Utilidades	Validación de matrículas y conversión de unidades.
`ZCL_AERONAVE_TEST_12`	Prueba	Prueba de la clase padre.
`ZCL_AVION_CARGA_TEST_12`	Prueba	Prueba del avión de carga.
`ZCL_AVISON_PASAJEROS_TEST_12`	Prueba	Prueba del avión de pasajeros.
`ZCL_TALLER_TEST_12`	Prueba	Prueba del taller.
`ZCL_AERO_UNIDADES_TEST_12`	Prueba	Prueba de las utilidades.
`ZCL_AEROFLOTA_TEST_12`	Prueba	Escenario completo que integra carguero y taller.

Tabla 1: Clases del proyecto

Las reglas de negocio que debe cumplir el sistema son:

- Cada aeronave debe pasar revisión cada 500 horas de vuelo. A partir de 450 horas el estado pasa a *revisión próxima*.
- Una aeronave con la revisión pendiente no puede registrar vuelos hasta que se registra la revisión, que pone a cero las horas desde la última revisión.
- El coste operativo base es de 1 000 por hora. El avión de carga añade 250 por hora de bodega y el de pasajeros 150 por hora de cabina.
- La mercancía perecedera (P) tiene un recargo de 200 y la peligrosa (M) de 500. La general (G) y cualquier tipo desconocido no tienen recargo (decisión D5).
- La carga de un avión es válida si es positiva y no supera su carga máxima.
- El taller solo admite intervenciones de tipo revisión (R), correctiva (C) o inspección (I), y siempre con horas positivas.
- Una matrícula española válida tiene 6 caracteres y empieza por `EC-`.
- El sistema lleva la cuenta del total de aeronaves creadas, incluidas las de las clases hijas.

1.3. Entorno de desarrollo

Utilizaremos Eclipse con ABAP Development Tools conectado a un sistema SAP. Las clases de prueba implementan la interfaz `IF_OO_ADT_CLASSRUN`, lo que permite ejecutarlas directamente con **F9** y ver el resultado en la consola de ADT sin necesidad de crear un programa ni una transacción.

Hay que tener en cuenta que, en ADT, guardar una clase no es lo mismo que activarla. Antes de ejecutar las pruebas conviene activar todos los objetos modificados con **Ctrl**+**Shift**+**F3**, ya que una clase de prueba no puede ejecutarse si alguna de las clases que utiliza está inactiva o tiene errores de sintaxis.

2. Diseño e implementación

2.1. Jerarquía de clases

La clase `ZCL_AERONAVE_12` contiene todo el comportamiento común. Las clases de carga y de pasajeros heredan de ella y solo añaden o redefinen lo que las diferencia. El taller y las utilidades son clases independientes.

```
ZCL_AERONAVE_12              (clase padre)
 +-- ZCL_AVION_CARGA_12      (hija: carga, bodega, mercancía)
 +-- ZCL_AVION_PASAJEROS_12  (hija: plazas, cabina, ocupación)

ZCL_TALLER_12                (independiente: intervenciones)
ZCL_AERO_UNIDADES_12         (independiente: métodos estáticos)
```

Las clases hijas se declaran como `FINAL`, porque no está previsto que nadie herede de ellas, mientras que la clase padre no lo es, ya que tiene que permitir la herencia.

2.2. Clase padre: ZCL_AERONAVE_12

2.2.1. Constantes y atributos

Las reglas numéricas se definen como constantes públicas, de modo que se pueden cambiar en un único sitio. Los atributos se reparten entre tres niveles de visibilidad:

```
CONSTANTS c_intervalo_revision TYPE i VALUE 500.
CONSTANTS c_umbral_aviso       TYPE i VALUE 450.
CONSTANTS c_coste_hora_base    TYPE i VALUE 1000.

PROTECTED SECTION.
  DATA matricula TYPE string.
  DATA modelo    TYPE string.

PRIVATE SECTION.
  DATA horas_totales        TYPE i.
  DATA horas_desde_revision TYPE i.
  CLASS-DATA total_aeronaves TYPE i.
```

- **Protegidos** (`matricula`, `modelo`): las clases hijas pueden leerlos directamente.
- **Privados** (`horas_totales`, `horas_desde_revision`): solo se modifican a través de los métodos de la clase padre, lo que garantiza que nadie puede saltarse las reglas de revisión.

- **Estático** (`total_aeronaves`): declarado con `CLASS-DATA`, se comparte entre todas las instancias de la clase y de sus hijas.

2.2.2. Constructor y contador de aeronaves

El constructor recibe la matrícula y el modelo, y de forma opcional las horas iniciales. Cada vez que se crea una aeronave, de cualquier tipo, se incrementa el contador estático:

```
METHOD constructor.
  matricula            = iv_matricula.
  modelo               = iv_modelo.
  horas_totales        = iv_horas_totales.
  horas_desde_revision = iv_horas_desde_revision.

  total_aeronaves += 1.
ENDMETHOD.

METHOD reiniciar_contador.
  total_aeronaves = 0.
ENDMETHOD.
```

El método estático `reiniciar_contador` existe para que las pruebas puedan partir siempre de cero, ya que el valor de un atributo estático se mantiene mientras dura la sesión.

2.2.3. Registro de vuelos y revisiones

El método `registrar_vuelo` rechaza las horas negativas y los vuelos de aeronaves con la revisión pendiente. Si el vuelo es válido, acumula las horas y devuelve `abap_true`. El método `registrar_vuelos_programados` reutiliza el anterior y se detiene en cuanto un vuelo es rechazado:

```
METHOD registrar_vuelo.
  IF iv_horas < 0 OR horas_desde_revision >= c_intervalo_revision.
    rv_ok = abap_false.
  ELSE.
    horas_totales        += iv_horas.
    horas_desde_revision += iv_horas.
    rv_ok = abap_true.
  ENDIF.
ENDMETHOD.

METHOD registrar_vuelos_programados.
  DO iv_vuelos TIMES.
    IF registrar_vuelo( iv_horas_por_vuelo ) = abap_false.
      EXIT.
    ELSE.
      rv_registrados += 1.
    ENDIF.
  ENDDO.
ENDMETHOD.
```

La comprobación se hace sobre las horas acumuladas *antes* del vuelo. Por eso una aeronave con 420 horas desde la revisión puede realizar un vuelo de 100 horas y terminar con 520, pero ya no podrá realizar el siguiente.

2.2.4. Estado de revisión

El estado se calcula a partir de las horas desde la última revisión, utilizando las constantes de la clase:

```
METHOD obtener_estado_revision.
  IF horas_desde_revision >= c_intervalo_revision.
    rv_estado = 'REVISION OBLIGATORIA'.
  ELSEIF horas_desde_revision >= c_umbral_aviso.
    rv_estado = 'REVISION PROXIMA'.
  ELSE.
    rv_estado = 'OPERATIVA'.
  ENDIF.
ENDMETHOD.

METHOD registrar_revision.
  horas_desde_revision = 0.
ENDMETHOD.
```

2.2.5. Ficha, vuelos hasta revisión y coste

La ficha de la aeronave se construye con una plantilla de texto (*string template*). El método `calcular_vuelos_hasta_revision` simula los vuelos con un

bucle `WHILE` y devuelve cuántos se pueden hacer antes de llegar a la revisión:

```
METHOD obtener_ficha.
  rv_ficha = |{ matricula } - { modelo } ({ horas_totales } h)|.
ENDMETHOD.

METHOD calcular_vuelos_hasta_revision.
  rv_vuelos = 0.
  IF iv_horas_por_vuelo <= 0.
    RETURN.
  ENDIF.
  DATA(lv_horas) = horas_desde_revision.
  WHILE lv_horas < 500.
    lv_horas  += iv_horas_por_vuelo.
    rv_vuelos += 1.
  ENDWHILE.
ENDMETHOD.

METHOD calcular_coste_operativo.
  rv_coste = iv_horas * c_coste_hora_base.
ENDMETHOD.
```

La clase ofrece además métodos de lectura (`obtener_horas_totales`, `obtener_horas_desde_revision`) y el método `obtener_datos`, que devuelve varios valores a la vez mediante parámetros `EXPORTING`.

2.3. Clase hija: ZCL_AVION_CARGA_12

La clase hereda de la aeronave con `INHERITING FROM` y añade la carga máxima como atributo privado. Su constructor llama primero al constructor de la clase padre con `super->constructor` y después guarda su propio dato:

```
METHOD constructor.
  super->constructor( iv_matricula = iv_matricula
                      iv_modelo    = iv_modelo ).
  carga_max_kg = iv_carga_max_kg.
ENDMETHOD.

METHOD calcular_coste_operativo.
  rv_coste = super->calcular_coste_operativo( iv_horas )
           + ( iv_horas * c_coste_bodega ).
ENDMETHOD.
```

El coste operativo se **redefine** (`REDEFINITION`): reutiliza el cálculo de la clase padre mediante `super->` y le suma el coste de bodega. Los dos métodos propios de la clase son el recargo por tipo de mercancía y la validación de la carga:

```
METHOD calcular_recargo_mercancia.
  CASE iv_tipo.
    WHEN 'P'.    rv_recargo = 200.
    WHEN 'M'.    rv_recargo = 500.
    WHEN OTHERS. rv_recargo = 0.
  ENDCASE.
ENDMETHOD.

METHOD validar_carga.
  IF iv_kg > 0 AND iv_kg <= carga_max_kg.
    rv_valida = abap_true.
  ELSE.
    rv_valida = abap_false.
  ENDIF.
ENDMETHOD.
```

El tipo de mercancía se declara con un tipo propio de la clase, `ty_tipo_mercancia`, definido como un carácter de longitud 1.

2.4. Clase hija: ZCL_AVION_PASAJEROS_12

La clase de pasajeros sigue el mismo patrón que la de carga, pero guarda el número de plazas y redefine dos métodos: el coste operativo, al que suma el coste de cabina, y la ficha, a la que añade las plazas:

```
METHOD constructor.
  super->constructor( iv_matricula = iv_matricula
                      iv_modelo    = iv_modelo ).
  plazas = iv_plazas.
ENDMETHOD.

METHOD obtener_ficha.
  rv_ficha = |{ super->obtener_ficha( ) } - { plazas } plazas|.
ENDMETHOD.

METHOD calcular_coste_operativo.
  rv_coste = super->calcular_coste_operativo( iv_horas )
           + ( iv_horas * c_coste_cabina ).
ENDMETHOD.
```

El porcentaje de ocupación comprueba primero los casos límite, de modo que nunca se divide entre cero y el resultado siempre queda entre 0 y 100:

```
METHOD calcular_ocupacion.
  IF plazas <= 0 OR iv_pasajeros <= 0.
    rv_porcentaje = 0.
  ELSEIF iv_pasajeros >= plazas.
    rv_porcentaje = 100.
  ELSE.
    rv_porcentaje = ( iv_pasajeros * 100 ) / plazas.
  ENDIF.
ENDMETHOD.
```

Hay que tener en cuenta que, en ABAP, la división entre enteros redondea el resultado en lugar de truncarlo: por ejemplo, 2 pasajeros de 3 plazas dan un 67 y no un 66.

2.5. Clase de taller: ZCL_TALLER_12

El taller guarda sus intervenciones en una **tabla interna**. Para ello define una estructura con los campos de cada intervención y un tipo de tabla basado en ella:

```
TYPES: BEGIN OF ty_intervencion,
         matricula TYPE string,
         tipo      TYPE c LENGTH 1,
         horas     TYPE i,
       END OF ty_intervencion.

TYPES ty_intervenciones TYPE STANDARD TABLE OF ty_intervencion
                        WITH EMPTY KEY.

CONSTANTS c_tipo_revision   TYPE c LENGTH 1 VALUE 'R'.
CONSTANTS c_tipo_correctiva TYPE c LENGTH 1 VALUE 'C'.
CONSTANTS c_tipo_inspeccion TYPE c LENGTH 1 VALUE 'I'.
```

El registro valida el tipo y las horas antes de añadir la fila a la tabla con `APPEND VALUE #( ... )`:

```
METHOD registrar_intervencion.
  IF iv_tipo <> c_tipo_revision
     AND iv_tipo <> c_tipo_correctiva
     AND iv_tipo <> c_tipo_inspeccion.
    rv_ok = abap_false.
    RETURN.
  ENDIF.

  IF iv_horas <= 0.
    rv_ok = abap_false.
    RETURN.
  ENDIF.

  APPEND VALUE #( matricula = iv_matricula
                  tipo      = iv_tipo
                  horas     = iv_horas ) TO intervenciones.
  rv_ok = abap_true.
ENDMETHOD.
```

Los métodos de consulta recorren la tabla con `LOOP AT`. Por ejemplo, las horas de una aeronave concreta se obtienen sumando solo las filas de su matrícula:

```
METHOD calcular_horas_aeronave.
  rv_horas = 0.
  LOOP AT intervenciones INTO DATA(ls_intervencion).
    IF ls_intervencion-matricula = iv_matricula.
      rv_horas = rv_horas + ls_intervencion-horas.
    ENDIF.
  ENDLOOP.
ENDMETHOD.
```

Con el mismo esquema se calculan las horas totales del taller (`calcular_horas_totales`) y el número de intervenciones de un tipo (`contar_por_tipo`). El número total de intervenciones se obtiene con la función `lines( )`.

2.6. Clase de utilidades: ZCL_AERO_UNIDADES_12

Esta clase solo contiene constantes y métodos estáticos (`CLASS-METHODS`), por lo que se usa sin crear ninguna instancia, con la sintaxis `clase=>metodo( )`:

```
METHOD validar_matricula.
  IF strlen( iv_matricula ) = 6
     AND substring( val = iv_matricula off = 0 len = 3 ) = 'EC-'.
    rv_valida = abap_true.
  ELSE.
    rv_valida = abap_false.
  ENDIF.
ENDMETHOD.

METHOD convertir_horas_a_minutos.
  rv_minutos = iv_horas * c_minutos_por_horas.
ENDMETHOD.
```

El orden de las condiciones es importante: ABAP deja de evaluar en cuanto el resultado está decidido, de modo que si la longitud no es 6 nunca se llega a ejecutar `substring`, que fallaría con textos de menos de 3 caracteres.

3. Pruebas

3.1. Estrategia de pruebas

Cada clase de negocio tiene su propia clase de prueba, y `ZCL_AEROFLOTA_TEST_12` combina varias clases en un escenario completo. Todas siguen el mismo esquema:

- Implementan `IF_OO_ADT_CLASSRUN` y escriben los resultados en la consola con `out->write( )`.
- Cada paso lleva un comentario con el resultado esperado, para poder compararlo a simple vista con la salida.
- Antes de empezar llaman a `reiniciar_contador( )`, para que el contador estático no arrastre aeronaves de ejecuciones anteriores (riesgo R4).
- Usan un método privado auxiliar que convierte los booleanos en los textos *Verdadero* y *Falso*:

```
METHOD texto_booleano.
  rv_texto = COND #( WHEN iv_valor = abap_true
                     THEN ‘Verdadero‘ ELSE ‘Falso‘ ).
ENDMETHOD.
```

Un paso típico de prueba tiene este aspecto:

```
" Paso 4. Esperado: Falso, la aeronave no puede volar
DATA(lv_ok) = lo_carguero->registrar_vuelo( 10 ).
out->write( |Paso 4 - registrar_vuelo( 10 ): { texto_booleano( lv_ok ) }|
     ).
```

3.2. Prueba de la clase padre

`ZCL_AERONAVE_TEST_12` crea tres aeronaves y recorre todos los métodos de la clase padre. A diferencia del resto de pruebas, escribe los booleanos directamente, por lo que en la consola aparecen como `X` (verdadero) o una línea vacía (falso).

Llamada	Resultado esperado
Total de aeronaves al empezar	0
Ficha de `a1` y `a2` (440 h desde revisión)	EC-ABC - A320 (12000 h) y EC-XYZ - ATR72 (12000 h)
Total de aeronaves	2
Estado de `a2`	OPERATIVA
Vuelos de 5 h y 10 h en `a2`	Fichas con 12005 h y 12015 h
`registrar_vuelo( 0 )`	X
`registrar_vuelo( -3 )`	Vacío (rechazado)
`registrar_vuelo( 50 )` desde 455 h	X (queda en 505 h)
Estado de `a2`	REVISION OBLIGATORIA
`registrar_vuelo( 1 )`	Vacío (revisión pendiente)
Horas totales de `a2`	12065
Revisión y nuevo estado	OPERATIVA
`registrar_vuelo( 10 )` y horas totales	X y 12075
`a1`: 10 vuelos programados de 100 h	1 (pasa de 440 a 540 h)
Estado y horas totales de `a1`	REVISION OBLIGATORIA y 12100
Vuelos hasta revisión: `a3` (120 h), `a1`, `a3` con 0 h	4, 0 y 0
`obtener_datos` de `a1`	EC-ABC A320 12100
Coste operativo de 10 h	10000
Total de aeronaves y tras reiniciar	3 y 0

Tabla 2: Resultados de `ZCL_AERONAVE_TEST_12`

3.3. Prueba del avión de carga

Se crea el carguero `EC-CGO` (B737F) con una carga máxima de 20 000 kg.

Paso	Llamada	Resultado esperado
1	Ficha heredada sin redefinir	EC-CGO - B737F (0 h)
2	`calcular_coste_operativo( 10 )`	12500 (10 x 1000 + 10 x 250)
3	Recargo mercancía G	0 (general)
4	Recargo mercancía P	200 (perecedera)
5	Recargo mercancía M	500 (peligrosa)
6	Recargo mercancía X	0 (desconocida, decisión D5)
7	`validar_carga( 15000 )`	Verdadero
8	`validar_carga( 20000 )`	Verdadero (justo en el límite)
9	`validar_carga( 20001 )`	Falso (supera la carga máxima)
10	`validar_carga( 0 )`	Falso (carga no positiva)

Tabla 3: Resultados de `ZCL_AVION_CARGA_TEST_12`

3.4. Prueba del avión de pasajeros

Se crea el avión `EC-PAX` (A320) con 150 plazas.

Paso	Llamada	Resultado esperado
1	Total de aeronaves	1 (la clase padre cuenta también a las hijas)
2	Ficha redefinida	EC-PAX - A320 (0 h) - 150 plazas
3	`calcular_coste_operativo( 10 )`	11500 (10 x 1000 + 10 x 150)
4	`calcular_ocupacion( 120 )`	80
5	`calcular_ocupacion( 200 )`	100 (más pasajeros que plazas)
6	`calcular_ocupacion( 0 )`	0
7	`registrar_vuelo( 100 )` y ficha	Verdadero y EC-PAX - A320 (100 h) - 150 plazas

Tabla 4: Resultados de `ZCL_AVISON_PASAJEROS_TEST_12`

El paso 7 demuestra que un método heredado sin redefinir (`registrar_vuelo`) funciona en la clase hija y que la ficha redefinida refleja las horas acumuladas por la clase padre.

3.5. Prueba del taller

Se crea el taller *Hangar Sevilla* y se registran varias intervenciones, válidas y no válidas.

Paso	Llamada	Resultado esperado
1	Intervenciones y horas al empezar	0 y 0
2	Registrar (EC-ABC, R, 40)	Verdadero
3	Registrar (EC-ABC, C, 15)	Verdadero
4	Registrar (EC-XYZ, I, 8)	Verdadero
5	Registrar (EC-XYZ, X, 5)	Falso (tipo no válido)
6	Registrar (EC-XYZ, I, 0)	Falso (horas no positivas)
7	Número de intervenciones	3
8	Horas totales	63 (40 + 15 + 8)
9	`contar_por_tipo` R, C, I y X	1, 1, 1 y 0
10	Horas de EC-ABC, EC-XYZ y EC-NNN	55, 8 y 0

Tabla 5: Resultados de `ZCL_TALLER_TEST_12`

3.6. Prueba de las utilidades

Paso	Llamada	Resultado esperado
1	`validar_matricula( EC-ABC )`	Verdadero
2	`validar_matricula( EC-AB )`	Falso (longitud 5)
3	`validar_matricula( XX-ABC )`	Falso (prefijo incorrecto)
4	`validar_matricula( ECABCD )`	Falso (sin guion)
5	`convertir_horas_a_minutos( 2 )`	120
6	`convertir_horas_a_minutos( 0 )`	0

Tabla 6: Resultados de `ZCL_AERO_UNIDADES_TEST_12`

3.7. Escenario completo: flota y taller

`ZCL_AEROFLOTA_TEST_12` reproduce un caso real: un carguero vuela hasta agotar su intervalo de revisión, queda bloqueado, pasa por el taller y vuelve a operar.

Paso	Llamada	Resultado esperado
1	Creación del carguero EC-CGO y del taller	Sin salida
2	10 vuelos programados de 100 h	5 (se detiene al llegar a 500 h)
3	Estado de revisión	REVISION OBLIGATORIA
4	`registrar_vuelo( 10 )`	Falso (no puede volar)
5	El taller registra (EC-CGO, R, 48)	Verdadero
6	Estado tras registrar la revisión	OPERATIVA
7	`registrar_vuelo( 10 )`	Verdadero
8	Horas totales del carguero	510
9	Horas de taller de EC-CGO	48
10	Total de aeronaves	1 (el taller no es una aeronave)

Tabla 7: Resultados de `ZCL_AEROFLOTA_TEST_12`

4. Incidencias detectadas y corrección

Al ejecutar las pruebas por primera vez varios resultados no coincidían con los esperados. La revisión del código permitió localizar y corregir los siguientes errores:

Clase	Problema	Corrección
`ZCL_AVION_CARGA_12`	El constructor no guardaba la carga máxima, así que toda carga era inválida.	Asignar `carga_max_kg` en el constructor.
`ZCL_AVION_CARGA_12`	`validar_carga` usaba `<` y rechazaba el valor límite.	Cambiar la comparación por `<=`.
`ZCL_AVION_CARGA_12`	Los recargos de G y P estaban intercambiados.	P = 200, M = 500 y el resto 0.
`ZCL_AVION_PASAJEROS_12`	El constructor no guardaba las plazas, lo que provocaba una división entre cero.	Asignar `plazas` y comprobar los límites antes de dividir.
`ZCL_AVION_PASAJEROS_12`	La ficha usaba `+`, que en ABAP es una suma numérica, y el número de plazas estaba escrito a mano.	Usar una plantilla de texto con el atributo `plazas`.
`ZCL_AERONAVE_12`	El formato de la ficha y los textos del estado no coincidían con los esperados.	Nuevo formato de ficha y textos en mayúsculas con las constantes.
`ZCL_AERONAVE_12`	El método se llamaba `reinicar_contador`.	Renombrarlo a `reiniciar_contador` en la clase y en todas las pruebas.
Pruebas de carga y pasajeros	Se instanciaban las clases sin el sufijo `_12`.	Usar el nombre completo de la clase.

Tabla 8: Errores encontrados y su solución

Quedan como mejoras pendientes, que no afectan a los resultados, sustituir el valor fijo 500 de `calcular_vuelos_hasta_revision` por la constante `c_intervalo_revision` y usar en `validar_matricula` las constantes `c_longitud_matricula` y `c_prefijo_espana`.

5. Conclusiones

¿Qué aporta la herencia? La clase padre concentra las reglas comunes a todas las aeronaves: horas de vuelo, revisiones y contador. Las clases de carga y de pasajeros solo añaden lo que las diferencia, de modo que una corrección en la lógica de revisiones se aplica automáticamente a todos los tipos de avión.

¿Para qué sirven la redefinición y `super`? La redefinición permite que cada clase hija adapte un comportamiento heredado, como el coste operativo o la ficha. Gracias a `super->` no se repite el cálculo de la clase padre, sino que se reutiliza y se amplía. Del mismo modo, `super->constructor` garantiza que la parte común del objeto siempre se inicializa correctamente.

¿Qué aporta la encapsulación? Las horas de vuelo son privadas y solo cambian a través de `registrar_vuelo` y `registrar_revision`. Así es imposible que otra clase ponga a cero las horas o haga volar un avión con la revisión pendiente saltándose las reglas.

¿Cuándo usar atributos y métodos estáticos? Cuando el dato o la operación pertenecen a la clase y no a un objeto concreto: el total de aeronaves se comparte entre todas las instancias, y las utilidades de matrícula y unidades no necesitan ningún estado propio.

¿Por qué probar con clases de consola? La interfaz `IF_OO_ADT_CLASSRUN` permite ejecutar cada escenario con F9 y comparar la salida con el resultado esperado. Las pruebas han permitido detectar errores que no producían fallos de sintaxis, como datos que el constructor no guardaba o reglas aplicadas al revés.

La práctica ha recorrido los conceptos fundamentales de ABAP orientado a objetos (clases, visibilidad, herencia, redefinición, miembros estáticos, constantes y tablas internas) y ha mostrado cómo una batería de pruebas ayuda a comprobar que todas las piezas encajan entre sí.